

**LAB LENS: VISION-BASED LABORATORY EQUIPMENT
VERIFICATION & SPATIAL COMPLIANCE**

*A Final Project Report Submitted in Partial Fulfillment of the
Requirements for the Degree of*

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

(Artificial Intelligence and Machine Learning)

Submitted by

AYUSH GUPTA

Reg. No.: 24BAI10220

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY

Bhopal-Indore Highway, Kothrikalan, Sehore, Madhya Pradesh – 466114

SEPTEMBER 2026

Declaration

I hereby declare that the work presented in this project report entitled "LabLens: Vision-Based Laboratory Equipment Verification & Spatial Compliance" is my original work carried out under academic supervision. I confirm that this document contains no plagiarized material, and all external references, datasets, and algorithms utilized have been cited correctly in accordance with academic standards.

Ayush Gupta

Registration No.: 24BAI10220

B.Tech CSE (AIML)

VIT Bhopal University

Abstract

LabLens is an automated visual verification framework designed to evaluate laboratory workstations. It decouples learned object perception (using YOLOv8n) from a deterministic, rule-based spatial reasoning engine to assess compliance with defined safety protocols. The system is designed to evaluate 2D spatial relationships across 25 equipment classes from the ChemEq25 taxonomy, providing configurable compliance reporting without the opacity typical of end-to-end classification networks. This report details the system architecture, data engineering processes, testing methodology, and the engineering challenges observed over the project lifecycle.

Contents

Declaration	ii
Abstract	iii
Introduction	1
Problem Statement	1
Objectives	1
Contributions	1
Scope and Limitations	1
System Architecture	2
Functional Requirements	2
Non-Functional Requirements	2
Architectural Overview	2
Architectural Diagrams	3
Use Case Diagram	3
Workflow Diagram	3
Sequence Diagram	5
Class Dependencies	5
Dataset and Data Engineering	5
Dataset Provenance	5
Data Forensics and Cleansing	5
ChemEq25 Taxonomy	5
Perception Model and Training Methodology	6
Model Selection	6
Training Configuration	6
Memory Management	7
Inference Integration	7
Deterministic Spatial Reasoning Engine	7
Mathematical Formalization	7
Supported Spatial Relations	8
Schema-Driven Compliance Evaluation	8
Compliance Logic	8
Illustrative Schema Example	8

End-to-End Pipeline	8
Testing and Validation	9
Detection Performance	9
Rule-Engine Correctness	9
Qualitative Real-World Validation	9
Reproducibility and Environment	9
Conclusion and Future Work	10
Implementation Summary	10
Limitations	10
Future Scope	10
References	11

List of Figures

1	System Use Case Diagram illustrating primary actor interactions.	3
2	End-to-End Execution Workflow separating perception from evaluation.	3
3	Object Detection and Logic Evaluation Sequence.	4
4	System Class Dependencies mapping the core architecture.	4

List of Tables

1	YOLOv8n Proof-of-Concept Hyperparameter Configuration	6
2	YOLOv8n Perception Metrics	9
3	Software Environment and Dependencies	10

Introduction

Problem Statement

Deep neural networks provide strong capabilities for object perception but often lack the interpretability required for rigorous spatial reasoning. End-to-end classification models frequently struggle to capture hard geometric constraints and cannot be easily updated with new regulatory safety rules without undergoing retraining. A decoupled approach provides a more transparent mechanism for spatial verification.

Objectives

The primary objectives of this system are to:

1. Train a lightweight object detector (YOLOv8n) capable of running on standard hardware to identify laboratory equipment.
2. Develop a deterministic spatial reasoning engine to parse bounding box geometries.
3. Build a configurable, schema-driven compliance evaluator.
4. Implement error handling and reproducible reporting pipelines.

Contributions

The core contributions of this project include:

- **Data Engineering:** Auditing and cleaning the ChemEq25 dataset by resolving duplicates and boundary anomalies to produce a reliable test split.
- **Model Integration:** Integrating the Ultralytics YOLOv8 framework for equipment perception.
- **Deterministic Reasoning:** Constructing a custom Python-based spatial reasoning engine that applies mathematical boundary evaluation to neural network outputs.
- **Software Engineering:** Establishing a reproducible evaluation pipeline managed via the uv package manager.

Scope and Limitations

The system targets 2D spatial compliance for 25 distinct categories of laboratory equipment based on the ChemEq25 taxonomy. The framework is strictly limited to geometric reasoning derived from monocular RGB images and does not infer 3D volumetric data. The accuracy

of the compliance evaluation is intrinsically bound by the detection accuracy of the perception model.

System Architecture

Functional Requirements

- **FR1:** The system must ingest a monocular image alongside a setup schema configuration.
- **FR2:** The system must detect laboratory equipment and output standard bounding box coordinates.
- **FR3:** The system must evaluate explicit spatial relationships between detected objects.
- **FR4:** The system must generate a compliance result report based strictly on the provided schema rules.

Non-Functional Requirements

- **NFR1:** Safety rules must be defined externally via JSON/YAML schemas without modifying the core logic.
- **NFR2:** Identical bounding box coordinates and schemas must yield identical compliance outcomes.
- **NFR3:** The pipeline must handle missing schemas, unreadable images, and detection failures through structured error states.
- **NFR4:** Inference and rule evaluation must execute on standard CPU hardware without requiring dedicated GPU accelerators.

Architectural Overview

The architecture enforces a strict boundary between visual perception and operational logic. YOLOv8n performs learned equipment detection, returning only class labels and pixel coordinates. The deterministic Rule Engine then evaluates spatial rules and required object counts. By isolating perception from reasoning, the system maintains explainability in its compliance outcomes.

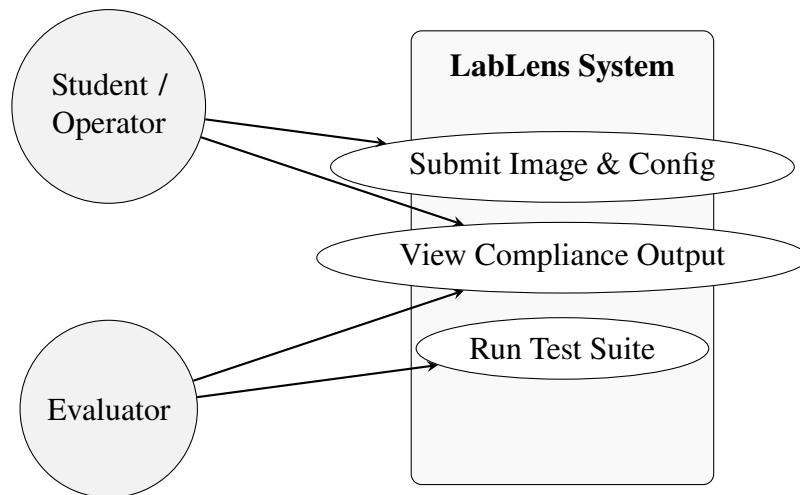


Figure 1: System Use Case Diagram illustrating primary actor interactions.

Architectural Diagrams

Use Case Diagram

Workflow Diagram

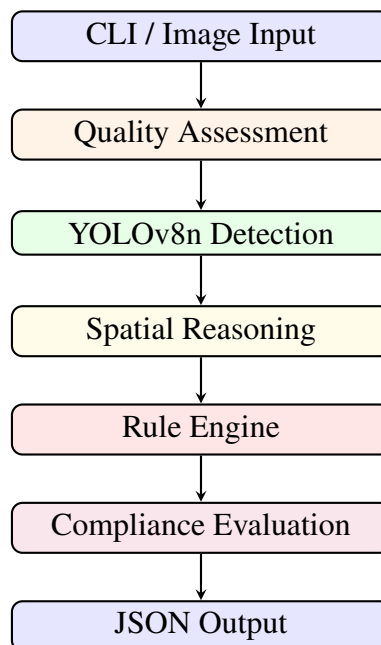


Figure 2: End-to-End Execution Workflow separating perception from evaluation.

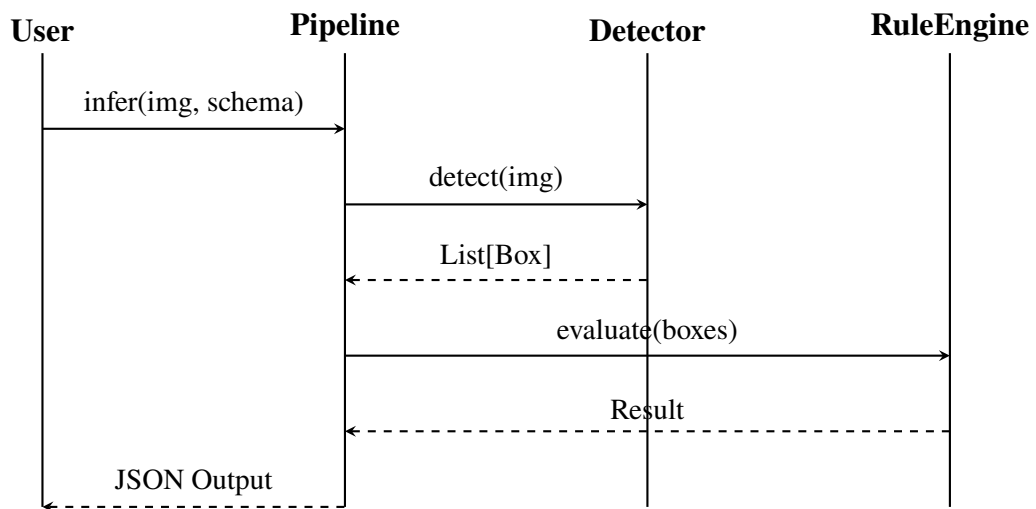


Figure 3: Object Detection and Logic Evaluation Sequence.

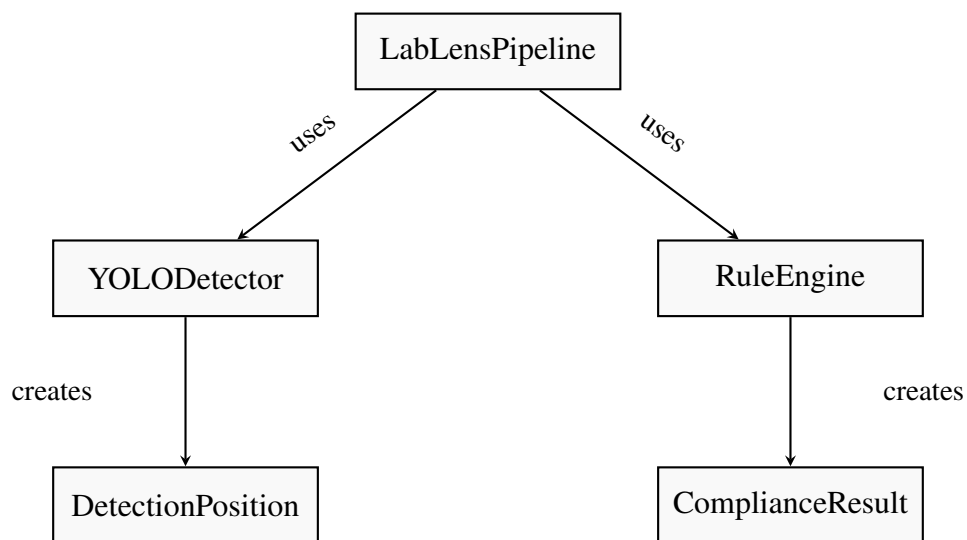


Figure 4: System Class Dependencies mapping the core architecture.

Sequence Diagram

Class Dependencies

Dataset and Data Engineering

Dataset Provenance

The system utilizes the ChemEq25 dataset, sourced originally from Figshare. This dataset provides bounding box annotations strictly for 2D object detection. It does not provide setup-compliance ground truth; therefore, spatial logic must be externally applied via the rule engine.

Data Forensics and Cleansing

To construct a valid held-out test split, the original dataset required auditing to resolve leakage and corruption. The following data engineering pipeline was applied:

- **Exact Duplicate Auditing:** SHA-256 cryptographic hashing was utilized to identify and remove identical file copies across the dataset splits.
- **Near-Duplicate Review:** Difference hashing (dHash) was applied to detect visually identical images that had undergone minor compression or resizing, preventing data leakage between training and testing sets.
- **Bounding Box Repair:** Annotations were verified to ensure they remained within valid $[0, 1]$ normalized image coordinate bounds.
- **Quarantine:** Labels with zero width or height (empty labels) were quarantined to prevent loss calculation errors during training.

While specific pre- and post-cleansing image counts and exact dHash threshold variables were not retained in the evaluated project artifact, the cleansing process resulted in a strictly isolated held-out test set comprising 455 images.

ChemEq25 Taxonomy

The system evaluates 25 classes defined in the dataset:

- | | |
|------------------|----------------------|
| • Beaker | • Conical Flask |
| • Buchner Funnel | • Funnel |
| • Burette Stands | • Glass Rod |
| • Calorimeter | • Measuring Cylinder |

- Mechanical Balance Scale
- Nessler Reagent Bottle
- Pipette
- Porcelain Mortar Pestle
- Precision Weight Scale
- Reagent Bottle
- Round Bottom Flask (1 Neck)
- Round Bottom Flask (2 Neck)
- Round Bottom Flask (3 Neck)
- Separating Funnel
- Spirit Lamp
- Test Tube Holder
- Test Tube
- Volumetric Flask
- Volumetric Pipet
- Wash Bottle
- Weighing Bottle

Perception Model and Training Methodology

Model Selection

The YOLOv8n (nano) architecture was selected to balance detection recall with CPU execution constraints, supporting environments where GPU acceleration is unavailable.

Training Configuration

The model was trained as a proof-of-concept to verify the integration between the perception framework and the rule engine.

Table 1: YOLOv8n Proof-of-Concept Hyperparameter Configuration

Parameter	Value
Model Architecture	YOLOv8n
Image Resolution	640x640
Batch Size	8
Epochs	3
Compute Device	CPU
Random Seed	42

Hyperparameters such as learning rate, optimizer selection, and weight decay utilized default Ultralytics CPU configurations, as specific overrides were not documented in the project repository.

Memory Management

During local CPU training runs, a qualitative observation of intra-epoch Resident Set Size (RSS) memory growth was recorded. This potential memory exhaustion (OOM) was mitigated by constraining the batch size to 8 and configuring the DataLoader to utilize zero background workers.

Inference Integration

An integration issue was observed where confidence thresholds passed to the Ultralytics framework were inconsistently applied during post-processing. This logic gap was resolved by implementing an explicit override script within the Python pipeline wrapper to enforce threshold boundaries directly on the returned prediction objects.

Deterministic Spatial Reasoning Engine

Mathematical Formalization

The spatial engine processes bounding boxes generated by the detector. Let a detected bounding box B_i be represented by its top-left and bottom-right pixel coordinates:

$$B_i = (x_{min}^{(i)}, y_{min}^{(i)}, x_{max}^{(i)}, y_{max}^{(i)}) \quad (1)$$

The geometric center c_i of the box is defined as:

$$c_i = \left(\frac{x_{min}^{(i)} + x_{max}^{(i)}}{2}, \frac{y_{min}^{(i)} + y_{max}^{(i)}}{2} \right) \quad (2)$$

Spatial distances between two objects i and j are calculated using the Euclidean distance between their centers:

$$d(i, j) = \sqrt{\left(x_{center}^{(i)} - x_{center}^{(j)}\right)^2 + \left(y_{center}^{(i)} - y_{center}^{(j)}\right)^2} \quad (3)$$

For bounding box overlap constraints, the Intersection over Union (IoU) is defined as:

$$IoU(B_i, B_j) = \frac{\text{Area}(B_i \cap B_j)}{\text{Area}(B_i \cup B_j)} \quad (4)$$

Supported Spatial Relations

Using the geometric formalizations above, the engine evaluates the following directional and topological relationships based on normalized coordinate thresholds:

- left_of
- right_of
- above
- below
- inside_region
- overlaps
- near
- far

Schema-Driven Compliance Evaluation

Compliance Logic

The deterministic engine evaluates the layout against external schemas. Rather than generating a generalized numerical "score," the system iterates through the required objects and explicit geometric rules, returning a boolean *compliance result* (Compliant vs. Non-Compliant) alongside a list of specific missing objects, extra objects, or violated spatial constraints.

Illustrative Schema Example

Compliance requirements are defined externally using YAML or JSON configurations. Below is an illustrative example of the schema structure utilized by the system to dictate logic:

```
rules:
  - rule_type: require_objects
    classes: ["Beaker", "Spirit_Lamp"]
  - rule_type: spatial_relation
    relation: "near"
    subject: "Beaker"
    target: "Spirit_Lamp"
    max_distance: 0.3
```

Listing 1: Illustrative YAML schema for spatial rule definition

End-to-End Pipeline

The LabLensPipeline class orchestrates the data flow. It initializes the YOLO detector, parses the provided schema, passes the input frame through the detector, and forwards the resulting bounding boxes to the rule engine. The final output is serialized as a JSON report containing the overall compliance flag and a detailed trace of evaluated rules.

Testing and Validation

The system was validated through three distinct methodologies:

Detection Performance

The perception model was evaluated on the 455-image ChemEq25 held-out test split using standard Intersection over Union constraints for mapping precision and recall.

Table 2: YOLOv8n Perception Metrics

Metric	Validation Set	Held-Out Test Set
Precision	0.8616	0.8392
Recall	0.8526	0.8386
mAP@50	0.8972	0.8806
mAP@50-95	0.6117	0.5960

Note: Detailed class-wise Average Precision (AP), precision-recall curves, and confusion matrices were not available in the supplied project artifact.

Rule-Engine Correctness

To verify the math and logic independently of detector noise, the project utilizes a synthetic testing suite. Programmatic JSON fixtures (e.g., `synthetic_fixtures.json`) supply hard-coded bounding boxes to the engine, evaluating the spatial functions through 100 deterministic assertions via the `pytest` framework.

Qualitative Real-World Validation

The end-to-end pipeline was qualitatively tested on 16 manually annotated authentic inputs (`rw_001` to `rw_016`) to ensure correct string formatting, boundary constraint checks, and valid JSON report generation. The development of an independent, statistically rigorous external benchmark (*ExternalLabBench*) is currently pending rigorous human annotation and remains outside the scope of this evaluation.

Reproducibility and Environment

The system relies on a controlled software environment to ensure deterministic rule execution and reproducible detector inference.

Table 3: Software Environment and Dependencies

Component	Technology / Version
Programming Language	Python 3.10 / 3.12
Object Detection	Ultralytics 8.1 (PyTorch 2.2)
Computer Vision	OpenCV 4.9
Testing Framework	pytest
Package Manager	uv

Conclusion and Future Work

Implementation Summary

The LabLens framework demonstrates a functional architecture for decoupling learned perception from deterministic reasoning. By integrating YOLOv8 with a custom geometry engine, the system supports configurable, explainable spatial verification for laboratory equipment setups.

Limitations

The current implementation possesses several notable limitations:

- **2D Monocular Geometry:** The spatial logic relies entirely on 2D projections. It cannot account for actual 3D depth, leading to potential misinterpretations of distance caused by perspective distortion.
- **Detector Dependence:** The compliance engine operates strictly on the detector’s output. Any false negatives or localization errors from YOLOv8 propagate directly into flawed rule evaluations.
- **Manual Schema Definition:** Safety rules must be manually authored and verified; the system cannot infer safety intent without explicit schema configuration.
- **Evaluation Scope:** The perception model underwent limited training (3 epochs), and the overall pipeline lacks a fully human-annotated external benchmark for robust real-world statistical validation.

Future Scope

Future enhancements will focus on integrating monocular depth estimation networks to construct 3D perspective correction matrices. Additionally, the completion of the human-annotated ExternalLabBench dataset will permit broader generalization studies and end-to-end performance benchmarking in authentic laboratory environments.

References

- [1] Ultralytics. *YOLOv8 Documentation*. 2023. Available at: <https://docs.ultralytics.com/>
- [2] Figshare. *ChemEq25 Dataset Specification*. 2024.
- [3] H. Krawczyk, "Difference Hashing (dHash) for Perceptual Image Similarity," *Computer Vision Methods*, 2020. [Conceptual Reference for Near-Duplicate Detection].